Mini Project Assessment

Business Case

You are working as a **Data Engineer** for **QuickCart**, a rapidly growing e-commerce and logistics platform.

The company currently faces:
- Slow dashboard performance
- Duplicate customer records
- Delayed reporting
- Pipeline failures
- High storage costs
- Difficulty processing clickstream logs
- Poor query performance on large datasets

The organization wants to build a scalable modern data platform using: Python, SQL, Pandas, , Airflow and modern ETL/ELT architecture.

Expected Deliverables

Students must submit:

1.SQL scripts (.sql)
2. Python notebooks/scripts (.py /.ipynb)
3. Airflow DAG (files/scripts)
4.Architecture diagram (word file/paint/onenote etc.)

DATASETS PROVIDED

1. customers.csv

customer_id
customer_name
email
city
signup_date

2. orders.csv

order_id
customer_id

product_id
order_date
quantity
unit_price
payment_status

3. products.csv

product_id
product_name
category
supplier_id
cost_price
selling_price

4. clickstream.csv

event_id
customer_id
event_type
page_url
event_timestamp
device_type

Dataset : given on the drive in a zipped folder

Section A - Data Engineering Theory

Q1. OLTP vs OLAP Architecture

The company currently stores transactional and analytical workloads in the same SQL database.

Tasks

1. Explain why this creates performance issues.
ANS: Storing both OLAP and OLTP databases in the same SQL results in performance issues as OLAPs queries which are usually large and slow, might block resources required for the many reads and writes of OLTPs, which require real time performance.

2. Compare OLTP and OLAP workloads.

ANS: OLTP workloads are typically write many read many workloads and can also update and delete records. These require lightweight faster queries and connections. OLAP work load, usually write once and do not modify records and are typical in large analytical

processes, with larger queries and historical data.

3. Suggest an improved architecture.

ANS: OLAP workloads and data could be stored and processed in a lakehouse in batch processes. OLTP workloads like the clickstream can be handled with streaming services.

4. Explain where:

-PostgreSQL /SQLServer/MySql

ANS: Customer, orders and products data

-Data warehouse

ANS: Historical clickstream data

-Data lake

ANS: All other typically unstructured data

should be used.

Q2. ACID vs BASE in Distributed Systems

QuickCart wants to store clickstream events from millions of users.

Tasks

1. Explain why BASE systems are preferred for clickstream systems.

ANS: BASE, which stands for Basically Available, meaning availability is the priority. Clickstreams are stochastic; hence, there might be a need for replication and later consistency resolution.

2. Which workloads require ACID compliance?

ANS: Workloads involving complex joins and aggregations where immediate consistency is required.

** Transactional data require compliance (single source of truth?)

3. Give examples where eventual consistency is acceptable.

ANS: Distributed clickstream data pipelines, where different nodes might report inconsistent data that is evolving in time.

Q3. Storage Formats & Querying

Tasks

1. Compare:

-CSV

Human readable file and suitable for small data exports. Structured

-JSON

Flexible schema, semi-structured, tzpical with API request. Human readable

-Avro

Binary type format usefully with streaming data and similar write heavy tasks

-Parquet

Columnar well compressed binary type, useful for large analytical queries

2. Why do analytical systems prefer columnar storage?

It is easy to query and aggregate columns by reading only data from required columns without having to load the entire dataset.

3. Explain avro vs json format.

Avro is schema-on-write type binary storage method. It compresses data efficiently. Preferred for large repetitive events data. JSON on the order hand is text like data, easily and universally readable, but with a lot of repeated characters hence can relatively bulky. JSON is schema-flexible.

4. Explain compression benefits in Parquet.

Parquet compresses each column accordingly based on the type of data in it. Eg repeated categorical data are encoded as unique integers.

Q4. Partitioning, Indexing

Tasks

1. Design a partitioning strategy for:

-orders table

Partitioned according to order date.

-clickstream data

Partioned with page_url

2. Suggest indexing strategy for:

-customer lookups

Customer_id (FKs) and city (geographical)

-order filtering

customer_id, order_date, product_id,
-joins
order_id, customer_id, product_id, event_id

3. Explain when indexes negatively affect performance.

Over indexing, increases the amount of writes, which is not suitable for high ingest work loads. Data modification processes are multiplied with more indexes.

Section B - SQL Engineering

Q1. Intermediate SQL Queries

Write SQL queries to:

1. Find top 10 customers by revenue.
2. Find month-over-month sales growth.
3. Find customers who ordered in consecutive months.
4. Find products never ordered.
5. Find revenue contribution percentage by category.

Q2. Advanced SQL

Tasks

1. Rank customers based on total revenue.
2. Find running total sales by month.
3. Find the highest selling product per category.
4. Find 7-day rolling average sales.

Q3. SQL Optimization Scenario

The following query takes 18 minutes:

```
SELECT c.customer_name,
 SUM(o.quantity * o.unit_price)
```

```
FROM orders o
JOIN customers c
ON o.customer_id = c.customer_id
WHERE order_date >= '2025-01-01'
GROUP BY c.customer_name;
```

Tasks

1. Identify possible bottlenecks.
    - Grouping by customer_name instead of customer_id
    - Equality constraint on text date. Should be converted to a proper timesta,p column
    - No index on order_date
    - Joining before aggregating
2. Suggest indexes.
    - customer_id
    - order_date
3. Explain execution plan analysis.
    - The statement 'EXPLAIN ANALYZE' actually runs the query and reports on real timings of the execution
4. Suggest partitioning improvements.
    - Range partition on order_date
5. Explain materialized views usefulness.
    - Materialized view physically store query results on the disk, replacing resource hungry recomputation with table scans.

Q4. Database Design

Design schema for:

1. Shipment tracking

    A. order_shipments
        a. post_id *PRIMARY KEY* *INDEX*
        b. order_id *REFERENCES orders(order_id)*
        c. customer_id *REFERENCES customers(customer_id)*
        d. order_weight
        e. postage_cost
        f. delivery_address
        g. dispatch _date *INDEX*
        h. delivery_date *INDEX*
        i. delivery_status

2. Product inventory

A. Product_inventory
    a. product*id REFERENCES products(product_id)*
    b. count
    c. supplier_id
    d. last_procured_date
    e. updated_at

3. Customer support tickets

A. tickets
    a. ticket_id PRIMARY KEY *INDEX*
    b. customer_id *REFERENCES customers(customer_id)*
    c. ticket_opened_date *INDEX*
    d. ticket_closed_date *INDEX*
    e. resolved_by *INDEX*
    f. support_category
    g. support_type
    h. support_details

Requirements:

• normalized design

• primary keys

• foreign keys

• indexing recommendations

Section C - Python + Pandas Engineering

Q1. Data Ingestion Pipeline

Using Python and Pandas:

Tasks

1. Read all datasets.

2. Validate schema consistency.

3. Detect duplicate customers.
4. Identify invalid records.

Q2. Data Cleaning & Transformation

Tasks

1. Standardize city names. (if any)
2. Convert timestamps correctly. (if any)
3. Handle missing values. (if any)
4. Remove corrupted rows.

Q3. Clickstream Analytics

Using clickstream data:

Tasks

1. Find the most visited pages.
2. Calculate session counts.
3. Find bounce rate.
4. Find mobile vs desktop traffic percentage.

Q4. Export Optimization

Tasks

1. Export analytical dataset into:
    - CSV
    - Parquet
2. Compare storage sizes.
3. Compare read performance.

Section D - ETL/ELT + Airflow

Q1. ETL vs ELT Architecture

Tasks

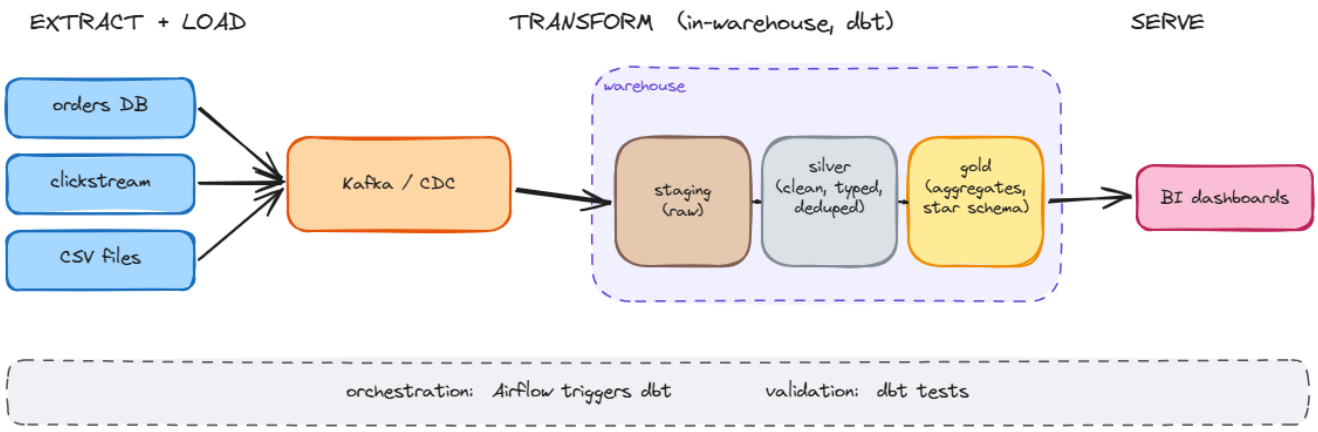
1. Compare ETL vs ELT.

In ETL, data is processed in a separate processing layer before it is loaded into the warehouse. These are usually schema-on-write situations. In ELT, the data lands in the warehouse first and is transformed later, usually using the warehouse's compute. These are most likely schema-on-read scenarios.

2. Explain why cloud warehouses prefer ELT.

ELT separates storage and compute. This means that even if everything is stored, transformation occurs only on demand, reducing transformation compute costs through the use of the so-called Elastic Compute. Cheap storage solutions make raw data retention possible. Reprocessing data is also easier, as the new processing functions only run on

3. Design modern ELT pipeline for QuickCart.



Q2. Airflow DAG Development

Create an Airflow DAG that:

Requirements

1. Runs daily at 2 AM
2. Extracts CSV files
3. Performs validation
4. Loads raw data into staging tables
5. Triggers transformation step
6. Sends failure email alerts

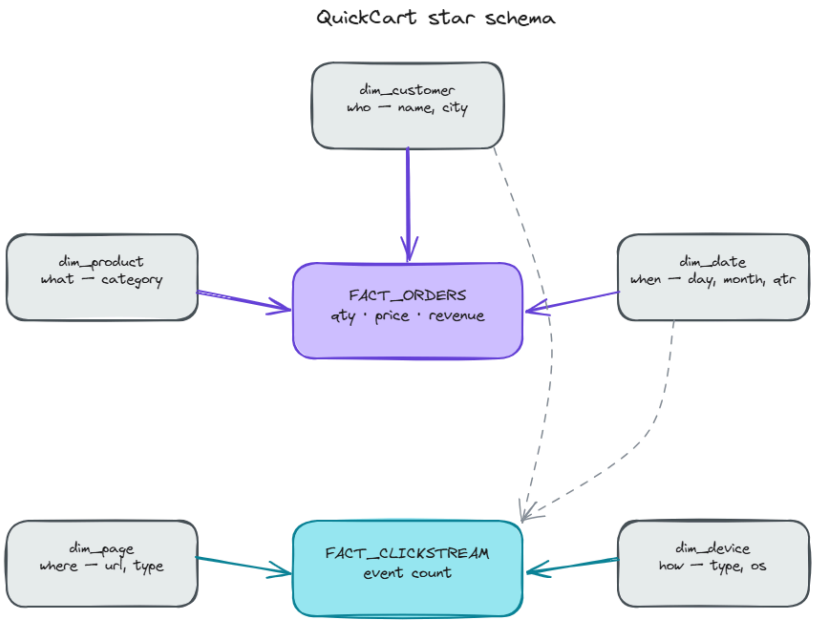
DAG must include:

- Retries
- retry delays
- task dependencies
- Logging
- scheduling
- sensors

Q3. Data Warehouse Modeling

Tasks

1. Design star schema.



2. Identify:

fact tables

- orders
- clickstream

dimension tables

ORDER

- Product
- Customer
- date

CLICKSTREAM

- Page
- device

Section E - Performance Engineering Case Study

Scenario

QuickCart processes:

• 50 million clickstream events/day

• 5 million orders/day

Problems:

• SQL queries are slow

• Pipelines fail frequently

• Dashboard refresh takes 3 hours •

Storage cost is increasing

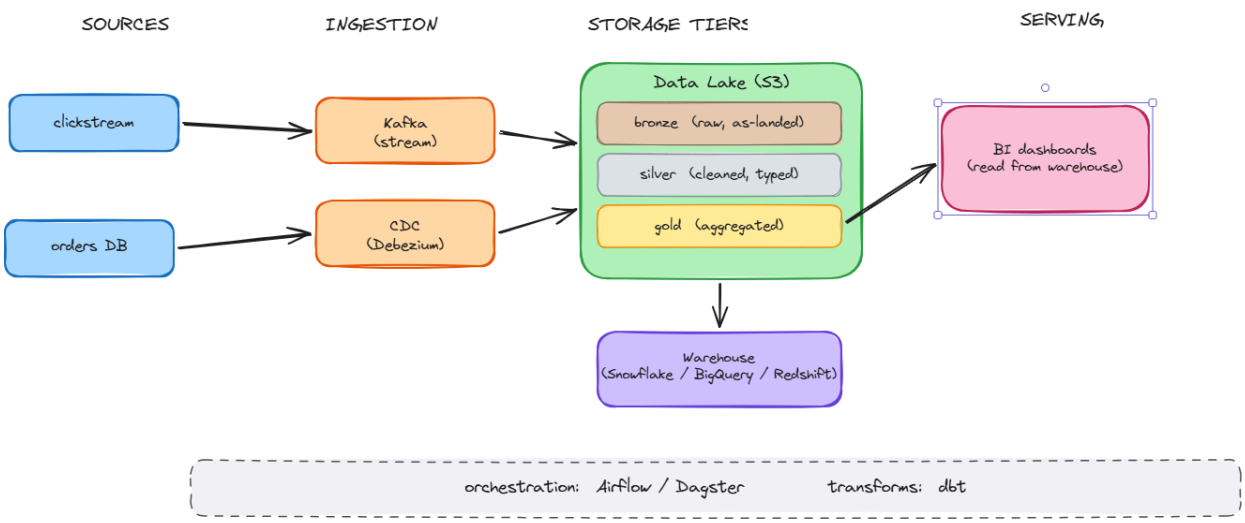
Tasks

Part 1 - Architecture Design

Design scalable architecture using:

- Database
- data lake

- warehouse
- Orchestration
- transformation layer



Part 2 - Optimization

Explain:

1. Partitioning strategy

Partition is a strategy for data storage used to reduce the number of rows scanned by dividing storage into parts. In this case, clickstream and orders data could be partitioned according to `event_date` and `order_date` columns, respectively because almost every query should filter on this. In this case, a daily partition would be appropriate. Tools like BigQuery and Iceberg are suitable for this.

2. Indexing strategy

OLTP data such as the orders database should be indexed on keys like the customer_id and other foreign keys and time-based data like order_date. Since B-Tree indexes are mostly unavailable in warehouses, clustering/sort_keys could be defined so that data queried together are stored physically close. Redshift does this well.

3. Compression strategy

Store data as Parquet with Zstandard, which is state of the art for ratio and speed. Data

should be stored by a low-cardinality column first and then tiered according to recency for storage tiers based on query ease and frequency. Fresh data queried frequently should be stored in warehouses, while historical data not frequently queried but must be saved can utilise cheap storage like S3 Glacier. There should be lifecycle policies that automatically ages data.

4. Query optimization

A. Filter early especially on partition columns (eg `event_date`, `order_date`) to reduce heavy work
B. Aggregrate data before joining when creating dashboards
C. Pre-aggregate recurring dashboard indicators in gold layer so dashboard don't always have to scan raw data.
D. Avoid `SELECT *` queries
E. Use materialized views
F. Use `EXPLAIN` to look for optimization tips

5. Incremental loading

When dashboards need updates, instead of processing all rows, the pipeline should process only what is new, since the last update.

6. Columnar storage benefits

Usually, only a few columns are needed for analytical queries. Querying raw data results in loading the entire data including irrelevant columns. Columnar storage on reads the required column, reducing drastically the data size to be scanned, increasing performance. Columnnar storage solutions like Parquet also compresses data well

Part 3 - Reliability

Explain:

1. Retry handling

Usually there can be failures withing the data pipeline. Retry handling is basically strategies to handle pipeline failure. Airflow manages this well. Example strategies are automatic retries with exponential backoff. Such strategies are categorised as Transient handling.

2. Failure alerts

Some failure alerts can be included when for example

1. A task exhausts retries, failing ultimately. In such scenario, an alert could be configured and sent by email or slack

2. Frehsness: Alert sent when data is not arriving at the proper time, avoiding the rish of outdated dashboards. Checks should be made to see if tables have not been updated in expected time
3. Volume anomaly: When row counts deviates sharply from expected trends.

3. Data quality validation

Tools like dbt could test for data integrity by checking if some data properties meet expectations. Such properties include

1. Schema: Check that the expected column types matches
2. Uniqueness: Primary keys and other unique keys are not duplicated, especially on retry
3. References: Every customer_id in facts table has a customer detail in customer dimension table
4. Range: Numeric data fits plausible constraints and timestamp is reasonable, For example, negative order quantity are not allowed and there cn not be a future clickstream event.
5. Categorical: payement_status for example is able to capture ‘Paid/paid’